

# CISCO INTERNSHIP ASSESSMENT

## NetSage AI

**Project GitHub link:** - <https://github.com/gadelohitareddy-cell/NetSage-AI-CISCO-INTERNSHIP>

**Project Frontend & Backend Working link:**

**Coding link:** [https://github.com/gadelohitareddy-cell/NetSage-AI-CISCO-INTERNSHIP/blob/main/CISCO%20CODING%20PART%20\(1\).mp4](https://github.com/gadelohitareddy-cell/NetSage-AI-CISCO-INTERNSHIP/blob/main/CISCO%20CODING%20PART%20(1).mp4)

**Backend Link:** [https://github.com/gadelohitareddy-cell/NetSage-AI-CISCO-INTERNSHIP/blob/main/BACKEND%20WORK\\_compressed.mp4](https://github.com/gadelohitareddy-cell/NetSage-AI-CISCO-INTERNSHIP/blob/main/BACKEND%20WORK_compressed.mp4)

**Project Working video:** [https://github.com/gadelohitareddy-cell/NetSage-AI-CISCO-INTERNSHIP/blob/main/PROJECT%20DEMO\\_compressed%20\(1\).mp4](https://github.com/gadelohitareddy-cell/NetSage-AI-CISCO-INTERNSHIP/blob/main/PROJECT%20DEMO_compressed%20(1).mp4)

## OVERALL PROJECT ANALYSIS

### NETSAGE AI

#### Chapter 1 – Introduction

- 1.1 Background
- 1.2 Need for the Project
- 1.3 Problem Statement
- 1.4 Objectives
- 1.5 Scope

#### Chapter 2 – Existing System

- 2.1 Existing Approach
- 2.2 Limitations

## Chapter 3 – Proposed System

### 3.1 Proposed Solution

### 3.2 Features

### 3.3 Advantages

### 3.4 Innovation

## Chapter 4 – System Requirements

### 4.1 Hardware Requirements

### 4.2 Software Requirements

### 4.3 Functional Requirements

### 4.4 Non-Functional Requirements

## Chapter 5 – System Design

### 5.1 System Architecture

### 5.2 Data Flow

### 5.3 Module Architecture

### 5.4 Folder Structure

## Chapter 6 – Implementation

### 6.1 Frontend

### 6.2 Backend

### 6.3 Diagnosis Service

### 6.4 Rule Validation

### 6.5 Human Review

### 6.6 History

### 6.7 Dashboard

### 6.8 API Implementation

## Chapter 7 – Testing

### 7.1 Testing Methodology

### 7.2 Test Cases

### 7.3 Results

## Chapter 8 – Results

### 8.1 Dashboard

### 8.2 Diagnosis

### 8.3 Diagnosis Result

### 8.4 Human Review

### 8.5 History

## Chapter 9 – Advantages and Limitations

## Chapter 10 – Future Scope

## Chapter 11 – Conclusion

## Chapter 12 – References

## **AI-Powered Cisco Packet Tracer Troubleshooting Assistant**

### **1. Abstract**

NetSage AI is a full-stack, AI-powered network troubleshooting assistant designed to help students, learners, and network engineers identify and resolve common networking problems encountered in Cisco Packet Tracer environments.

Network troubleshooting generally requires users to examine network symptoms, Cisco IOS command outputs, topology information, configurations, and OSI-layer behavior. For beginners especially, identifying the actual source of a fault can be difficult and time-consuming.

NetSage AI provides a structured troubleshooting workspace where users can submit:

- Network symptoms
- Cisco show command output
- Network topology information
- Fault category or concept

The system processes this information through a FastAPI-based backend containing diagnostic services and deterministic rule-validation mechanisms. It generates a structured diagnosis containing:

- Probable root cause
- Confidence level
- Severity
- OSI layer
- Supporting evidence
- Recommended verification commands
- Recommended Cisco IOS configuration changes
- Step-by-step troubleshooting guidance
- Rule-validation results

The React-based frontend presents this information through an interactive dashboard. A human review mechanism allows reviewers to Accept, Edit, or Reject AI-generated recommendations, ensuring that automated suggestions remain subject to human oversight.

The system also maintains diagnostic history and provides dashboard-level statistics, filtering, searching, and export capabilities.

---

## **2. Introduction**

### **2.1 Background**

Computer networks consist of multiple interconnected devices such as:

- Routers
- Switches
- Servers
- Wireless access points

A small configuration error in any of these components can prevent communication across the network.

Cisco Packet Tracer is widely used for learning and simulating networking concepts. Students use it to configure devices and practice protocols such as:

- VLAN
- DHCP
- DNS
- OSPF
- NAT
- ACL
- Wireless authentication
- Routing and switching

However, troubleshooting these problems requires a systematic understanding of network protocols and Cisco IOS commands.

## **2.2 Need for the Project**

Traditional troubleshooting requires the user to:

1. Understand the reported network symptom.
2. Identify the possible OSI layer involved.
3. Examine device configuration.
4. Execute appropriate Cisco IOS commands.
5. Interpret command output.
6. Compare the output with expected network behavior.
7. Identify the probable root cause.
8. Apply a suitable configuration change.
9. Verify whether the issue has been resolved.

For beginners, this process can be confusing.

NetSage AI attempts to simplify this workflow by providing a guided and auditable troubleshooting process.

---

### 3. Problem Statement

Cisco Packet Tracer users frequently encounter networking issues such as:

- VLAN mismatches
- Incorrect trunk configuration
- OSPF adjacency failures
- DHCP address exhaustion
- Incorrect DNS configuration
- ACL blocking legitimate traffic
- NAT configuration errors
- Wireless authentication failures
- Routing problems
- Incorrect IP addressing

Identifying the exact cause of these problems manually can be difficult, particularly for learners who are unfamiliar with Cisco IOS commands and systematic troubleshooting methodologies.

Existing troubleshooting approaches often require users to search through documentation, forums, configuration guides, or manually interpret command output.

Therefore, there is a need for a system that can:

- Interpret network symptoms.
- Analyze Cisco command output.
- Associate problems with networking concepts.
- Identify probable root causes.
- Provide evidence for the diagnosis.
- Recommend appropriate verification commands.
- Suggest possible remediation commands.
- Maintain diagnostic history.
- Allow human experts to validate AI recommendations.

NetSage AI addresses this requirement by providing a structured network troubleshooting assistant.

---

## 4. Objectives

The major objectives of NetSage AI are:

### 4.1 Automated Diagnosis

To analyze network symptoms and Cisco CLI output and identify probable network configuration problems.

### 4.2 Guided Troubleshooting

To provide users with a systematic sequence of troubleshooting steps instead of requiring them to determine the entire troubleshooting process independently.

### 4.3 Root Cause Identification

To identify the most likely cause of a network problem based on the available evidence.

### 4.4 Evidence-Based Recommendations

To explain why a particular problem is suspected by presenting relevant evidence from the submitted information.

### 4.5 Cisco IOS Recommendations

To provide suitable Cisco IOS verification and remediation commands.

### 4.6 Human Oversight

To allow reviewers to:

- Accept recommendations
- Edit recommendations
- Reject recommendations

### 4.7 Diagnostic History

To store previous diagnosis records and allow users to search and filter them.

### 4.8 Dashboard Analytics

To provide statistics regarding:

- Number of diagnoses
- Severity distribution
- Fault categories
- Review status

## 5. Scope of the Project

The scope of NetSage AI includes network troubleshooting scenarios that can be represented using symptoms, topology information, and Cisco CLI output.

The system focuses on areas such as:

- VLAN configuration
- Switching
- Routing
- OSPF
- DHCP
- DNS
- ACL
- NAT
- Wireless networking

The project is primarily intended for:

- Networking students
- Cisco Packet Tracer learners
- Network engineering trainees
- Instructors
- Lab environments
- Network troubleshooting practice

The current implementation focuses on structured diagnostic analysis rather than directly connecting to physical Cisco devices.

---

## 6. Existing System

In a conventional network troubleshooting environment, users generally follow a manual approach.

Existing workflow

Network Problem





Observe Symptoms



Run Cisco Commands



Read CLI Output



Search Documentation



Identify Possible Cause



Modify Configuration



Test Again

Limitations

### **6.1 Time Consuming**

Users may need to execute many commands before finding the source of a problem.

### **6.2 Requires Networking Knowledge**

A user must understand Cisco IOS commands, protocols, and networking concepts.

### **6.3 Difficult for Beginners**

Students may know that a network is not working but may not know which command to execute next.

### **6.4 Manual Interpretation**

CLI output must be manually interpreted and compared with expected behavior.

### **6.5 Lack of Structured Evidence**

Traditional troubleshooting does not always provide a clear record explaining:

- What was detected?
- Why was it detected?
- What evidence supports the conclusion?

- What action was recommended?

## 6.6 No Built-in Review Workflow

There may not be a dedicated mechanism for an instructor or network expert to validate recommendations.

---

## 7. Proposed System

NetSage AI introduces a structured troubleshooting environment.

Proposed workflow

User



Submit Network Symptoms



Add CLI Output



Add Topology Notes



Select Fault Category



FastAPI Backend



Diagnostic Service



Rule Validation



Structured Diagnosis



React Dashboard



Root Cause + Evidence



Verification Commands



Remediation Commands



Human Review



Accept / Edit / Reject



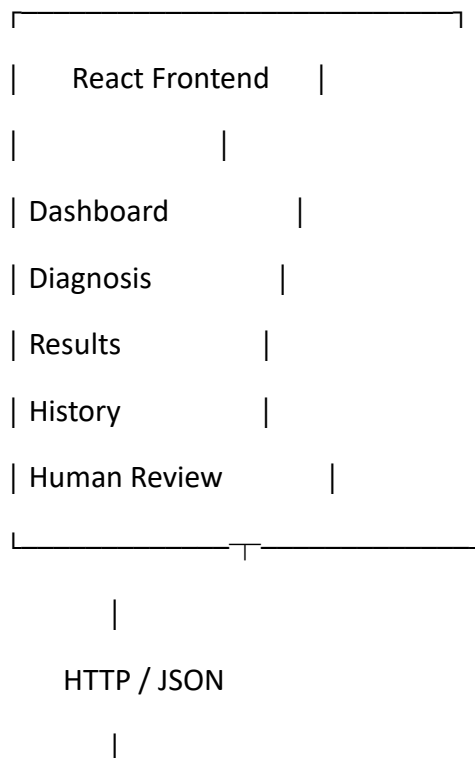
Diagnostic History

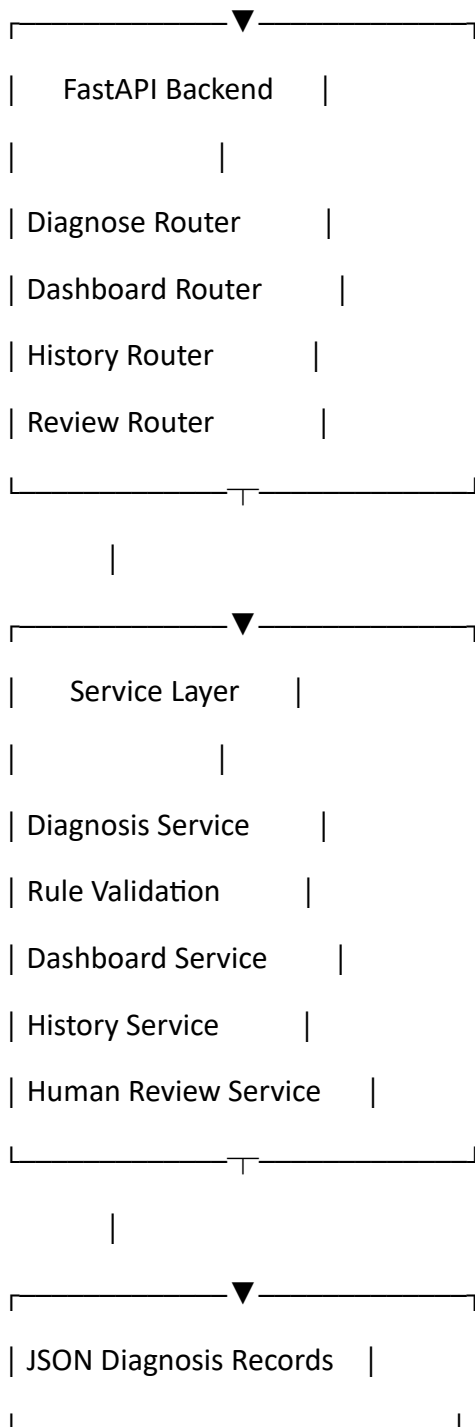
The system combines automated diagnostic reasoning, deterministic validation, and human review.

---

## 8. System Architecture

NetSage AI follows a client-server full-stack architecture.





---

## 9. Technology Stack

| Layer      | Technology |
|------------|------------|
| Frontend   | React 19   |
| Build Tool | Vite       |

| Layer               | Technology          |
|---------------------|---------------------|
| Routing             | React Router        |
| Styling             | Tailwind CSS        |
| Icons               | Lucide React        |
| Backend             | Python              |
| API Framework       | FastAPI             |
| Data Validation     | Pydantic            |
| Server              | Uvicorn             |
| Persistence         | JSON                |
| Package Management  | npm / pip           |
| Code Quality        | Oxlint              |
| Network Environment | Cisco Packet Tracer |

---

## 10. Frontend Architecture

The frontend is developed using React 19 and Vite.

The frontend provides an interactive user interface for submitting diagnosis requests and viewing the results.

Major frontend components

### 10.1 Dashboard

The dashboard provides a high-level view of diagnostic information.

It displays metrics such as:

- Total diagnoses
- Diagnosis distribution
- Severity distribution
- Review status
- Network concept distribution

### 10.2 Diagnosis Page

The diagnosis page allows users to enter:

- Network symptom
- CLI output
- Topology notes
- Fault category

The information is submitted to the backend through the diagnosis API.

### 10.3 Diagnosis Result Page

The result page displays:

- Root cause
- Confidence
- Severity
- OSI layer
- Evidence
- Rule-validation results
- Verification commands
- Remediation commands
- Troubleshooting steps

### 10.4 History Page

The history page displays previous diagnoses.

Users can filter records based on:

- Status
- Concept
- Severity
- Search keywords

### 10.5 Human Review Page

The review page allows authorized users or reviewers to evaluate recommendations.

Available actions include:

- Accept

- Edit
  - Reject
- 

## **11. Backend Architecture**

The backend is developed using Python and FastAPI.

FastAPI handles HTTP requests and routes them to the appropriate service.

The backend is divided into multiple logical layers.

### **11.1 Router Layer**

The router layer receives API requests.

Major routers include:

- Diagnose Router
- Dashboard Router
- History Router
- Human Review Router

### **11.2 Schema Layer**

Pydantic schemas define the structure of:

- Request data
- Response data
- Diagnosis records
- Review requests

This provides structured data validation.

### **11.3 Service Layer**

The service layer contains the main application logic.

Services include:

Diagnosis Service

Responsible for processing network troubleshooting requests.

Rule Validation Service

Validates diagnosis results against predefined troubleshooting rules.

Dashboard Service

Generates metrics and statistical information.

History Service

Retrieves and filters historical diagnosis records.

Human Review Service

Processes reviewer decisions.

---

## **12. Diagnosis Workflow**

The diagnosis process can be divided into several stages.

### **Step 1: User Input**

The user enters:

- Network symptom
- Cisco CLI output
- Topology information
- Fault category

Example:

Symptom:

PCs in VLAN 10 cannot communicate with the server.

CLI Output:

show vlan brief

Topology:

Two switches connected through a trunk.

Fault Category:

VLAN

### **Step 2: Request Validation**



The FastAPI backend validates the request using Pydantic models.

Invalid or incomplete information can be identified before processing.

### Step 3: Diagnostic Processing

The diagnosis service examines the provided information.

It identifies patterns related to known networking problems.

### Step 4: Rule Validation

The system applies deterministic troubleshooting rules.

For example, if the input indicates a VLAN mismatch, the system can validate whether the provided evidence supports the diagnosis.

### Step 5: Diagnosis Generation

The system creates a structured diagnosis.

The result can contain:

- Root cause
- Confidence
- Severity
- OSI layer
- Evidence
- Recommended command
- Remediation commands
- Troubleshooting guide

### Step 6: Result Presentation

The frontend receives the JSON response and displays the information in a readable format.

### Step 7: Human Review

A reviewer can evaluate the recommendation and:

- Accept it
- Edit it
- Reject it

### Step 8: History Storage

The diagnosis and review information can be stored as a diagnosis record for future reference.

---

### **13. Diagnostic Output**

One of the important aspects of NetSage AI is its structured diagnosis output.

A diagnosis may contain the following fields:

Root Cause

The most likely reason for the network problem.

Confidence

Indicates how strongly the system supports the diagnosis.

Severity

Represents the potential impact of the identified issue.

Example:

- Low
- Medium
- High
- Critical

OSI Layer

Identifies the relevant networking layer.

For example:

- Layer 2 – Data Link
- Layer 3 – Network
- Layer 4 – Transport
- Layer 7 – Application

Evidence

Provides information supporting the diagnosis.

Verification Command

Provides a Cisco IOS command that can be executed to confirm the diagnosis.

Example:

show vlan brief

Remediation Command

Provides possible Cisco IOS configuration changes.

Troubleshooting Guide

Provides a step-by-step procedure for resolving the problem.

---

## **14. Supported Troubleshooting Scenarios**

### **14.1 VLAN Problems**

Possible issues include:

- Incorrect VLAN assignment
- VLAN mismatch
- Incorrect access-port configuration
- Trunk-related problems

Useful commands may include:

show vlan brief

show interfaces switchport

show interfaces trunk

---

### **14.2 OSPF Problems**

Potential issues include:

- Incorrect network statements
- Area mismatch
- Interface configuration problems
- Adjacency failures

Useful commands include:

show ip ospf neighbor

show ip ospf interface

show ip route

---

### 14.3 DHCP Problems

Potential issues include:

- DHCP pool exhaustion
- Incorrect network configuration
- Incorrect default gateway
- DHCP service configuration errors

Useful commands include:

`show ip dhcp pool`

`show ip dhcp binding`

`show running-config`

---

### 14.4 DNS Problems

Possible problems include:

- Incorrect DNS server address
- DNS resolution failure
- Incorrect client configuration

The system can guide the user toward verification of DNS-related settings.

---

### 14.5 ACL Problems

Possible issues include:

- Incorrect ACL rule
- Incorrect source/destination
- Incorrect protocol/port
- Incorrect ACL application

Useful commands include:

`show access-lists`

`show running-config`

---

## 14.6 NAT Problems

Possible issues include:

- Incorrect inside/outside interface configuration
- Incorrect translation rule
- Incorrect ACL associated with NAT

Useful commands include:

show ip nat translations

show ip nat statistics

show running-config

---

## 14.7 Wireless Problems

Possible issues include:

- Incorrect SSID
- Authentication mismatch
- Incorrect security configuration
- Incorrect wireless settings

The system can identify potential configuration problems based on the submitted symptoms and evidence.

---

## 15. Rule Validation

Rule validation is an important component of the project.

Instead of simply producing a recommendation, the system checks whether the diagnosis is consistent with predefined troubleshooting rules.

For example:

Input Evidence



Expected Pattern



Rule Matching



Validation Result



Diagnosis Confidence

This provides an additional layer of verification.

Benefits

- Improves consistency.
- Reduces unsupported recommendations.
- Provides explainable evidence.
- Makes troubleshooting easier to audit.
- Supports human review.

---

## 16. Human Review Workflow

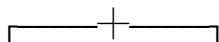
NetSage AI does not rely entirely on automated recommendations.

The system includes a human-in-the-loop workflow.

AI Recommendation



Human Reviewer



Accept Edit Reject

Accept

The reviewer confirms that the recommendation is appropriate.

Edit

The reviewer modifies the recommendation before approving it.

Reject

The reviewer determines that the recommendation is incorrect or unsuitable.

Importance

Human review helps ensure:

- Accuracy
  - Accountability
  - Expert oversight
  - Safer configuration recommendations
  - Better learning outcomes
- 

## **17. Dashboard**

The dashboard provides a centralized overview of the system.

It can display:

Total Diagnoses

Total number of diagnosis requests processed.

Severity Distribution

Displays how many problems belong to different severity levels.

Concept Distribution

Shows the distribution of problems across:

- VLAN
- OSPF
- DHCP
- DNS
- ACL
- NAT
- Wireless

Review Status

Shows the number of:

- Accepted diagnoses

- Edited diagnoses
- Rejected diagnoses
- Pending reviews

This allows users and reviewers to understand the overall troubleshooting activity.

---

## **18. Diagnostic History**

The history module maintains previously generated diagnoses.

Users can search and filter records using:

- Status
- Concept
- Severity
- Free-text search

For example:

Search: OSPF

Results:

Diagnosis 01 – OSPF adjacency failure

Diagnosis 02 – OSPF area mismatch

Diagnosis 03 – OSPF interface issue

This makes the system useful not only as a troubleshooting tool but also as a learning and reference system.

---

## **19. API Documentation**

NetSage AI provides REST APIs through FastAPI.

### **19.1 GET /**

Provides basic API metadata and service status.

### **19.2 GET /health**

Used to verify that the backend service is running.



### 19.3 POST /api/diagnose

Used to submit a new network troubleshooting request.

Input

The request can contain:

- Symptom
- CLI output
- Topology notes
- Fault category

Output

The API returns a structured diagnosis.

---

### 19.4 GET /api/dashboard

Returns dashboard metrics and diagnostic distributions.

---

### 19.5 GET /api/history

Returns historical diagnosis records.

It supports filters such as:

status

concept\_tag

severity

search

---

### 19.6 POST /api/review

Used to submit a human review decision.

Possible actions include:

Accept

Edit

Reject

---

## 20. Data Flow

The overall data flow is:

User



React Interface



HTTP Request



FastAPI API



Pydantic Validation



Diagnosis Service



Rule Validation



Structured Diagnosis



JSON Response



React Result Page



Human Review



History

---

## 21. Database / Data Persistence

The current implementation uses JSON-based local persistence.

Diagnosis records are stored in JSON format.

Advantages of the current approach include:

- Simple implementation
- Easy debugging
- No database server required
- Suitable for prototype/hackathon development
- Easy portability

However, JSON persistence has limitations for large-scale deployment, which can be addressed through a production database in future versions.

---

## 22. Project Folder Structure

```
.
├── backend/
│   ├── app/
│   │   ├── routers/
│   │   │   ├── diagnose.py
│   │   │   ├── dashboard.py
│   │   │   ├── history.py
│   │   │   └── review.py
│   │   │
│   │   └── schemas/
│   │       └── ...
│   │
│   └── services/
│       ├── diagnosis_service.py
│       ├── dashboard_service.py
│       └── history_service.py
```

```
| | | └─ review_service.py
| | |
| | └─ data/
| | | └─ diagnosis_records.json
| | |
| | └─ main.py
| |
| └─ requirements.txt
|
└─ src/
  └─ components/
  └─ layouts/
  └─ pages/
    └─ Dashboard
    └─ Diagnosis
    └─ Result
    └─ History
    └─ Review
  └─ services/
  └─ package.json
└─ README.md
```

---

## 23. Installation Requirements

### Hardware Requirements

A basic development system is sufficient.

#### Recommended:

- Modern dual-core or better processor
- 4 GB RAM minimum
- 8 GB RAM recommended
- At least 5 GB free storage
- Internet connection for package installation

#### Software Requirements

- Node.js 18+
  - npm
  - Python 3.10+
  - Cisco Packet Tracer
  - Modern web browser
  - Git
- 

## 24. Installation Procedure

### Step 1: Clone Repository

```
git clone <repository-url>
```

```
cd <project-directory>
```

### Step 2: Install Frontend Dependencies

```
npm install
```

### Step 3: Install Backend Dependencies

```
python -m pip install -r backend/requirements.txt
```

---

## 25. Running the Frontend

Start the Vite development server:

```
npm run dev
```

Vite will display the local development URL.

Typically:

<http://localhost:5173>

The frontend can then be opened in a browser.

---

## 26. Running the Backend

From the project root:

```
uvicorn app.main:app --app-dir backend --reload
```

The backend will normally be available at:

<http://localhost:8000>

FastAPI also provides interactive API documentation at:

<http://localhost:8000/docs>

---

## 27. Configuration

The frontend can communicate with a different backend URL using:

VITE\_API\_BASE\_URL

This makes it possible to deploy the frontend and backend separately.

---

## 28. User Interface Modules

The application consists of several major screens.

### 28.1 Dashboard Screen

Displays:

- Overall metrics
- Diagnostic distribution
- Severity information
- Review information

Screenshot:

Add your Dashboard screenshot here.

[Insert Dashboard Screenshot]

---

## 28.2 Diagnosis Screen

Allows the user to submit network troubleshooting information.

Screenshot:

[Insert Diagnosis Screenshot]

---

## 28.3 Diagnosis Result Screen

Displays:

- Root cause
- Confidence
- Evidence
- OSI layer
- Severity
- Verification commands
- Remediation commands
- Troubleshooting steps

Screenshot:

[Insert Diagnosis Result Screenshot]

---

## 28.4 Human Review Screen

Allows the reviewer to:

- Accept
- Edit
- Reject

recommendations.

Screenshot:

[Insert Human Review Screenshot]

---

## 28.5 History Screen

Displays previously generated diagnoses and provides search/filter functionality.

Screenshot:

[Insert History Screenshot]

---

## 29. Testing

Testing is performed to ensure that the frontend, backend, APIs, diagnosis logic, and review workflow operate correctly.

### 29.1 Functional Testing

The following functions should be tested:

| Test Case        | Expected Result                   |
|------------------|-----------------------------------|
| Open dashboard   | Dashboard loads successfully      |
| Submit diagnosis | Diagnosis is generated            |
| Invalid input    | Validation error is displayed     |
| View result      | Structured diagnosis is displayed |
| Open history     | Previous records are displayed    |
| Search history   | Matching records appear           |
| Filter severity  | Relevant records appear           |
| Accept review    | Status changes appropriately      |
| Edit review      | Modified recommendation is saved  |
| Reject review    | Recommendation is marked rejected |
| API health check | Backend returns healthy status    |

---

## 30. Advantages

### 30.1 Faster Troubleshooting

Users receive structured diagnostic guidance instead of manually investigating every possible cause.

### 30.2 Beginner Friendly



The step-by-step workflow helps learners understand troubleshooting methodology.

### 30.3 Explainable Results

The system provides evidence and rule-validation information instead of presenting only a final answer.

### 30.4 Cisco-Oriented

The recommendations are designed around Cisco IOS troubleshooting commands and configuration.

### 30.5 Human Oversight

Reviewers can validate AI recommendations before they are accepted.

### 30.6 Historical Learning

Previous diagnosis records can be used as a reference for future troubleshooting.

### 30.7 Modular Architecture

Frontend, backend, routing, services, and schemas are separated, making the application easier to maintain.

---

## 31. Limitations

The current implementation has several limitations.

### 31.1 Local JSON Storage

JSON persistence is suitable for a prototype but is not ideal for large-scale multi-user deployment.

### 31.2 Limited Device Integration

The current system does not directly connect to physical Cisco routers or switches.

### 31.3 Packet Tracer File Analysis

Automatic analysis of Cisco Packet Tracer topology files is not currently implemented.

### 31.4 Authentication

The current implementation does not include a complete user authentication and authorization system.

### 31.5 AI/LLM Integration

The current diagnostic workflow primarily uses structured logic and deterministic rule validation. Full LLM-assisted analysis is planned for future development.

### 31.6 Limited Collaboration

Real-time collaboration and review queues are not currently implemented.

---

## 32. Future Scope

NetSage AI can be expanded significantly.

### 32.1 Production Database

Replace JSON storage with databases such as:

- PostgreSQL
- MySQL
- MongoDB

### 32.2 User Authentication

Implement:

- User accounts
- Login
- Role-based access
- Admin accounts
- Reviewer accounts

### 32.3 LLM Integration

Integrate an LLM to provide:

- Natural-language explanations
- More flexible symptom interpretation
- Context-aware troubleshooting
- Explainable reasoning

The deterministic rules can continue to act as a validation layer.

### 32.4 Packet Tracer Topology Analysis

Allow users to upload Packet Tracer topology files and automatically extract:

- Devices
- Interfaces

- IP addresses
- VLANs
- Connections
- Routing configurations

### 32.5 Device Inventory

Support automatic device inventory import.

### 32.6 Real-Time Collaboration

Multiple users could collaborate on the same troubleshooting case.

### 32.7 Notification System

Reviewers could receive notifications when a diagnosis requires approval.

### 32.8 PDF Reports

Generate professional troubleshooting reports in PDF format.

### 32.9 Ticketing Integration

Integrate with IT service-management platforms.

### 32.10 Cloud Deployment

Deploy the application using cloud platforms and implement:

- CI/CD
- Automated testing
- Monitoring
- Scalable backend infrastructure

---

## 33. Security Considerations

Security should be considered when the system moves beyond a local prototype.

Important security mechanisms include:

- Authentication
- Authorization
- Role-based access control
- Input validation

- API security
- Secure environment variables
- HTTPS
- Logging and monitoring
- Protection against malicious CLI input
- Secure storage of user data

If AI/LLM services are integrated, sensitive network configuration information should also be handled carefully.

---

### **34. Educational Impact**

NetSage AI can be particularly useful in networking education.

Students can use the platform to learn:

- Cisco IOS commands
- Network troubleshooting
- OSI model concepts
- VLAN configuration
- Routing
- DHCP
- ACL
- NAT
- OSPF
- Wireless networking

Instead of simply providing a solution, the system can help students understand:

Problem → Evidence → Diagnosis → Verification → Remediation

This encourages systematic troubleshooting rather than trial-and-error configuration.

---

### **35. Project Innovation**

The key innovation of NetSage AI is not simply generating a network troubleshooting answer.

It combines several components into one workflow:

Network Symptoms

+

CLI Output

+

Topology Context

↓

Automated Diagnosis

↓

Rule Validation

↓

Evidence

↓

Cisco Verification Commands

↓

Remediation Guidance

↓

Human Review

↓

Historical Record

This makes the system structured, explainable, auditable, and educational.

---

### 36. Use Case

Example: VLAN Troubleshooting

A student reports:

"PC connected to Switch A cannot communicate with PC connected to Switch B."

The student provides:

Fault Category:

## VLAN

### Topology:

Two switches connected through a trunk.

### CLI Output:

```
show vlan brief
```

```
show interfaces trunk
```

NetSage AI processes the information and may identify a possible VLAN/trunk configuration issue.

The system presents:

Root

Possible VLAN/trunk configuration mismatch.

OSI

Layer 2 – Data Link.

### Evidence:

Provided CLI output indicates that the expected VLAN is not correctly carried across the connection.

### Verification Command:

```
show interfaces trunk
```

### Recommended Action:

#### Verify:

- VLAN existence
- Access-port assignment
- Trunk configuration
- Allowed VLAN list

The reviewer can then accept, edit, or reject the recommendation.

---

## 37. Expected Outcome

After implementation, NetSage AI provides a centralized environment where users can:

1. Submit network problems.
  2. Provide Cisco CLI evidence.
  3. Specify topology information.
  4. Select a troubleshooting category.
  5. Receive structured diagnostic findings.
  6. Understand the probable root cause.
  7. Identify the relevant OSI layer.
  8. Examine supporting evidence.
  9. Verify the diagnosis using Cisco commands.
  10. Apply recommended remediation.
  11. Submit the recommendation for human review.
  12. Track the diagnosis in history.
  13. Analyze troubleshooting statistics through the dashboard.
- 

### **38. Conclusion**

NetSage AI provides a structured approach to Cisco Packet Tracer troubleshooting by combining a modern web interface with a FastAPI-based diagnostic backend.

The system addresses common difficulties faced by networking learners and engineers by converting raw troubleshooting information into structured diagnostic findings.

The platform provides:

- Automated diagnosis
- Rule-based validation
- Evidence-based recommendations
- Cisco IOS verification commands
- Remediation guidance
- Dashboard analytics
- Diagnostic history
- Human review
- Search and filtering

The human-in-the-loop design is particularly important because it ensures that automated recommendations can be reviewed and validated by a knowledgeable person.

Although the current version is primarily a prototype, its architecture provides a foundation for future capabilities such as LLM integration, Packet Tracer topology analysis, production databases, authentication, real-time collaboration, cloud deployment, and automated report generation.

Overall, NetSage AI aims to transform network troubleshooting from a trial-and-error activity into a guided, explainable, and auditable workflow.

---

### **39. References**

You can include references such as:

1. Cisco Systems – Cisco IOS documentation.
  2. Cisco Packet Tracer documentation and learning resources.
  3. FastAPI documentation.
  4. React documentation.
  5. Vite documentation.
  6. Python documentation.
  7. Pydantic documentation.
  8. Tailwind CSS documentation.
  9. Networking fundamentals and OSI model references.
  10. Relevant research papers on AI-assisted network troubleshooting.
-





# NETSAGE AI

AI-Powered Cisco Packet Tracer Troubleshooting Assistant

Tagline: *Diagnose. Validate. Resolve. Learn.*

Presented by: NetSage AI Team — Technology: React | FastAPI | Python | Cisco Packet Tracer

Made with Dribbble

## The Problem

Network troubleshooting in Cisco Packet Tracer is time-consuming and challenging for learners:

- Root-cause identification from symptoms is difficult.
- Cisco CLI output can be dense and hard to interpret.
- Beginners rely on trial-and-error and don't know which command to run next.
- Troubleshooting spans multiple OSI layers and protocols without a structured workflow.
- No built-in method to validate or verify recommended fixes.

Common trouble areas: VLAN · OSPF · DHCP · DNS · ACL · NAT · Wireless





## Introducing NetSage AI

NetSage AI converts symptoms, Cisco CLI evidence and topology notes into a structured, explainable troubleshooting workflow — accelerating diagnosis while preserving human oversight.



### User Input

Symptoms, CLI output, topology context, fault category.



### NetSage Engine

Diagnostic reasoning + deterministic rule validation.



### Output

Root cause, evidence, verification & remediation commands.



### Human Review

Accept · Edit · Reject— creates auditable records.

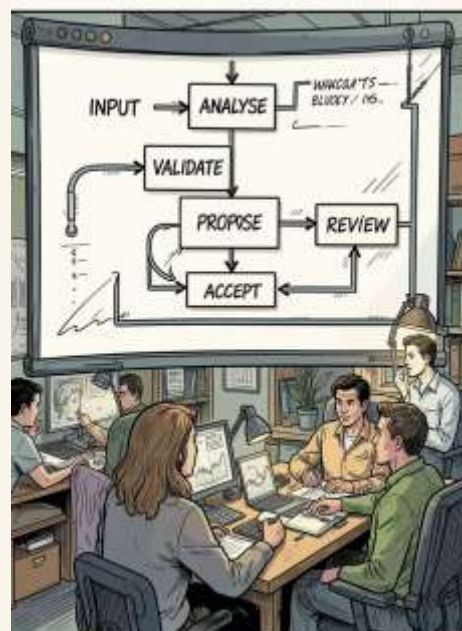


## Troubleshooting Workflow

Streamlined process designed for clarity and repeatability:



Final output includes: Root Cause, Confidence, Severity, OSI Layer, Evidence, Verification command, Remediation commands, Step-by-step guide.



## Key Features



### Smart Diagnosis

Probabilistic identification of likely configuration faults with ranked hypotheses.



### Evidence-Based Results

Explicit CLI excerpts tied to each diagnosis for auditability and teaching moments.



### Rule Validation

Deterministic checks confirm whether remediation steps resolve the issue.



### Cisco IOS Guidance

Verification and remediation command snippets formatted for immediate use in Packet Tracer.



### Human-in-the-Loop

Reviewer workflow to Accept, Edit or Reject — keeps experts in control.



### History & Analytics

Searchable diagnostic history and dashboards for pattern discovery and pedagogy.



## Supported Network Problems

NetSage AI targets common Packet Tracer scenarios across protocol domains and OSI layers:

### Areas & Example Faults

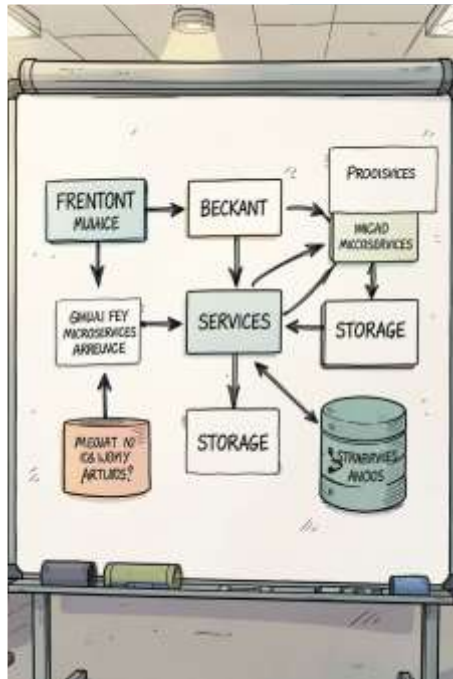
- VLAN—mismatch, access/trunk misconfig
- OSPF—adjacency failures, area/type mismatch
- DHCP—pool exhaustion, binding issues
- DNS—resolution failures, misconfigured servers
- ACL—unintended blocks, ordering mistakes
- NAT—translation failures, incorrect interfaces
- Wireless—authentication and association problems

### Troubleshooting Commands

Common verification commands  
NetSage highlights:

- show vlan brief
- show interfaces trunk
- show ip ospf neighbor
- show ip dhcp binding
- show accesslists
- show ip nat translations





## System Architecture

Purpose-built, full-stack architecture for interactive diagnosis and audit:

## Frontend

React 19, Vite, Tailwind— dashboard, input forms, review workflows.

## API Layer

FastAPI + Uvicorn—diagnosis routing, validation, history APIs.

## Diagnostic Services

Rule validation engine, evidence extractor, CLI parsers, LLM components (future).

## Persistence

JSONbased records for rapid prototyping; extensible to databases for production

## Human -in-the-Loop & Explainability

NetSage blends automated reasoning with expert review to build trust and provenance.



## Why human review matters

- Validates AI recommendations before any config changes.
- Prevents unsafe or inappropriate actions on devices.
- Provides expert feedback that improves future diagnoses.
- Creates auditable records for training and compliance.

Every diagnosis documents: Root Cause → Evidence → Validation checks → Verification command → Remediation steps.





## Results & Impact

### Students

Guided troubleshooting, faster learning curve, clearer understanding of Cisco IOS commands.

### Trainers

Review student work, identify repeating mistakes, tailor lessons to common faults.

### Network Engineers

Faster triage, evidence-based recommendations, searchable historical cases for reference.

Impact summary: Faster troubleshooting · Better learning outcomes · Explainable, auditable recommendations · Human oversight retained



## Future Scope & Conclusion

### Planned Enhancements

- LLM-assisted network analysis for richer natural-language diagnostics.
- Packet Tracer topology file parsing and automated evidence extraction.
- Production-grade storage, authentication and rolebased access.
- Real-time collaboration, PDF reports and ticketing integration.
- Cloud deployment with CI/CD and scalable services.

### Conclusion

NetSage AI transforms trial-and-error Packet Tracer troubleshooting into a guided, explainable and auditable workflow. Diagnose smarter, troubleshoot faster, and learn better.

Next step: pilot deployment with instructors and lab classes to refine rules, UX and case coverage.